

23m left

1. Question 1

Kalpna has a satellite phone, which communicates with other phones through orbiting satellites rather than terrestrial cell sites. The satellite phone's keypad is made up of alphanumeric characters that look like the figure below:



Language C++20 Environment Autocomplete Ready

```
1 > #include <bits/stdc++.h> ...
9
10 /*
11  * Complete the 'noOfPossibleWords' function below.
12  *
13  * The function is expected to return an INTEGER.
14  * The function accepts following parameters:
15  * 1. STRING_ARRAY arr
16  * 2. STRING S
17  */
18
19 int noOfPossibleWords(vector<string> arr, string S) {
20
21 }
22
23 > int main() ...
```

Test Results Custom Input

19m left



ALL



1

2

3



To call a contact, Kalpana has to tap the key repeatedly for each required letter. For example, if she wishes to call "rani", she has to press the keys as: r-777 a-2 n-66 i-444.

This is proving to be very cumbersome to Kalpana, and she has come up with an idea. She is pruning her contact list, such that any number combination she types will result in only one match from her contact list. Her contact list contains contacts made up entirely of lowercase letters from the English alphabet.

After pruning the list, Kalpana is left with N contacts. Can you help her with a program to check if the list is pruned enough that any number combination will match with only one contact? She'll provide you a string S which is the number combination, and you have to figure out how many contacts from the contact list can be mapped to S .

Language

Environment

4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24



Type here to search



19m left



ALL



1

2

3

After pruning the list, Kalpana is left with N contacts. Can you help her with a program to check if the list is pruned enough that any number combination will match with only one contact? She'll provide you a string S which is the number combination, and you have to figure out how many contacts from the contact list can be mapped to S .

Complete the *noOfPossibleWords* function, which has 3 input parameters and returns an integer *result*, which is the number of contacts matching with S . The 3 input parameters are:

1. An integer N that indicates the number of contacts in the contact list.
2. An array of strings *arr* which is the contact list.
3. A number S represented as a string, that contains digits 2-9, which is the number combination.

Input Format:

The first line of input contains the integer



Type here to search



18m left



ALL



1

2

3

Sample Input 0:

```
2
fiba
bida
3422
```

Sample Output0:

```
1
```

Explanation 0:

There are 2 contacts in the contact list:
fiba and bida

The string S: 3422

Here, key-strokes pattern will be:

f - 3 **i** - 4 **b** - 2 **a** - 2

b - 2 **i** - 4 **d** - 3 **a** - 2

So, only 1 contact fiba matches with S.
Hence 1 is printed as output.

Sample Input 1:

Language

Enviro

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24



Type here to search



18m left



ALL



1

2

3

The first line of input contains the integer N , the number of contacts in the contact list.

Each of the next N lines contains a single string representing one contact.

The last line of input contains the string S that consists of digits 2-9, which is the number combination.

Output Format:

Single line of output contains the number of contacts that can be mapped to the string S .

Constraints:

- i. $1 \leq N \leq 1000$
- ii. Length of each contact $< 10^6$ characters
- ii. $1 \leq \text{Length of number combination } S \leq 1000$

Sample Input 0:

```
2
fiba
bida
```



Type here to search



34m left

▼ Input Format For Custom Testing

The first contains an integer, n , the number of pixels in the image. Each of the next n lines contains an integer $pixel[i]$.

▼ Sample Case 0**Sample Input For Custom Testing**

STDIN		FUNCTION
5	→	pixel[] size n =
5		
3	→	pixel = [3, 3,
3, 3, 3]		
3		
3		
3		
3		

Sample Output

6

Explanation

It is optimal to perform 2 operations on the first and the last pixels, and 1 operation on the second and the next to the last pixels to get [1, 2, 3, 2, 1].

► Sample Case 1

Language C++14

Environment

```
1 > #include <bits/stdc++>
9
10 /*
11  * Complete the 'getMinOperations' function.
12  *
13  * The function is expected to return an integer.
14  * The function accepts an array of integers 'pixel' and its size 'n' as input.
15  */
16
17 long getMinOperations(int pixel[], int n) {
18
19 }
20
21 > int main() {
```

Test Results

MacBook

34m left

ExampleGiven $n = 5$ and $pixel = [1, 2, 5, 2, 1]$.

It is optimal to perform two operations to get the *pixel* array $[1, 2, 3, 2, 1]$ which is clear. Hence the answer is 2.

ALL

**Function Description**

Complete the function *getMinOperations* in the editor below.



getMinOperations takes the following arguments:

2

int pixel[n]: the pixel values of the image

3

Returns

long int: the minimum number of operations to make the array *pixel* clear

Constraints

- $1 \leq n \leq 3 * 10^5$
- $1 \leq pixel[i] \leq 10^9$

► Input Format For Custom Testing**▼ Sample Case 0****Sample Input For Custom Testing**

Language C++14

Environment

```
1 > #include <bits/stdc++>
9
10 /*
11  * Complete the 'getMinOperations' function.
12  *
13  * The function must return a long integer.
14  * The function must not use any global variables.
15  */
16
17 long getMinOperations(int pixel[], int n) {
18
19 }
20
21 > int main() {
```

Test Results

34m left

2. Question 2

ALL

i

✓

2

3

Some developers are working on clarification of an image with n pixels, where the value of the j^{th} pixel is represented by $pixel[j]$.

An image is clear if the array $pixel$ has a padding of some number of 0s (possibly none) in the prefix and the suffix and the non-zero part of the array $pixel$, say from index i to index j , starts from $pixel[i] = 1$, strictly increases for some k in steps of 1, and then strictly decreases in steps of 1 such that $pixel[j] = 1$.

For example, the $pixel$ arrays $[0, 0, 1, 2, 3, 2, 1]$, $[1, 2, 1]$, $[1]$, $[0, 0, 0, 1, 2, 3, 4, 3, 2, 1, 0]$ are clear.

In a single operation, the developers can reduce any pixel value by 1.

Given n pixels of an image and an array $pixel$, find the minimum number of operations required to make the array $pixel$ clear.

Example

Given $n = 5$ and $pixel = [1, 2, 5, 2, 1]$.

Language C++14

Environment

```
1 > #include <bits/stdc++  
9  
10 /*  
11 * Complete the 'ge  
12 *  
13 * The function is  
14 * The function ac  
15 */  
16  
17 long getMinOperat  
18  
19 }  
20  
21 > int main() ...
```

Test Results

Mac

34m left

▼ Input Format For Custom Testing

The first contains an integer, n , the number of pixels in the image. Each of the next n lines contains an integer $pixel[i]$.

▼ Sample Case 0**Sample Input For Custom Testing**

STDIN		FUNCTION
5	→	pixel[] size n =
5		
3	→	pixel = [3, 3,
3, 3, 3]		
3		
3		
3		
3		

Sample Output

6

Explanation

It is optimal to perform 2 operations on the first and the last pixels, and 1 operation on the second and the next to the last pixels to get [1, 2, 3, 2, 1].

► Sample Case 1

Language C++14

Environment

```
1 > #include <bits/stdc++>
9
10 /*
11  * Complete the 'getMinOperations' function.
12  *
13  * The function is expected to return an integer.
14  * The function accepts an array of integers 'pixel' and its size 'n' as input.
15  */
16
17 long getMinOperations(int pixel[], int n) {
18
19 }
20
21 > int main() {
```

Test Results

MacBook

33m left



ALL



3

Complete the function
`getLongestSequence` in the editor below.
`getLongestSequence` has the following
parameter(s):

`int matrix[n][m]`: a 2-D array of
integers

Returns

`int`: an integer value denoting the
length of the longest non-decreasing
sequence.

Constraints

- $1 \leq n \leq 1000$
- $1 \leq m \leq 1500$
- $1 \leq \text{matrix}[i][j] \leq 10^9$

► Input Format For Custom Testing

► Sample Case 0

► Sample Case 1



Type here to search



33m left



ALL



3

One of the optimal sequences is :

- Take {2} from the 1st column.
- Take {4} from the 3rd column.
- Take {7} from the 4th column.

Hence, we get the non-decreasing sequence = [2, 4, 7], having a length of 3, which is the maximum possible.

Function Description

Complete the function `getLongestSequence` in the editor below. `getLongestSequence` has the following parameter(s):

int matrix[n][m] : a 2-D array of integers

Returns

int: an integer value denoting the length of the longest non-decreasing sequence.

Constraints



Type here to search



SS
HY

24

F S
1
5 7 8
13 14 15
20 21 22
27 28 29

2024

T F S
1 2 3
8 9 10
4 15 16 17
1 22 23 24
8 29 30 31



33m left



ALL



3

3. Question 3

You are given a grid *matrix* of integers consisting of n rows and m columns. Find the length of the longest non-decreasing sequence by taking at-most one element from each column without changing the order of columns. Note: The columns from which the elements are chosen should also be strictly increasing.

Example

Given $n = 3, m = 4$, *matrix* =

2	6	3	5
8	9	4	6
5	8	9	7

One of the optimal sequences is :